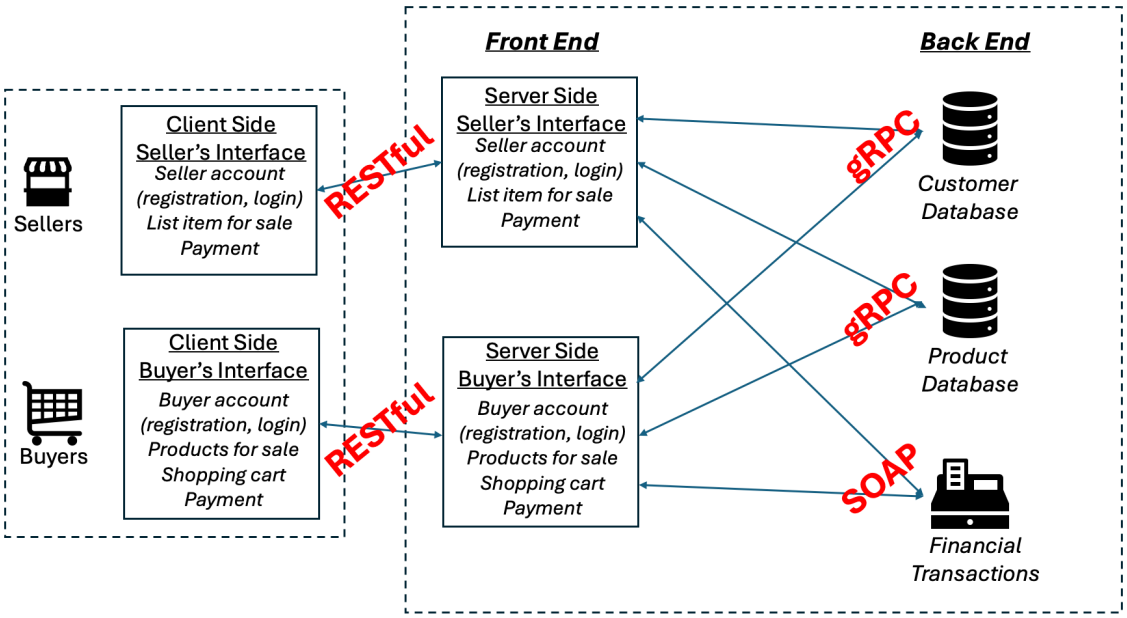
CSCI/ECEN 5673: Distributed Systems
Spring 2026

Programming Assignment Three

Due 11:59PM, Monday, March 30, 2026

Goal: The goal of this programming assignment is to extend the online marketplace system you developed in Programming Assignment One and Two as follows:

- Design and implement a rotating sequencer atomic broadcast protocol
- Replicate the customers database over five servers using your rotating sequencer atomic broadcast protocol
- Replicate the products database over five servers using Raft
- Replicate the server-side seller's interface over four servers
- Replicate the server-side buyer's interface over four servers



Characteristics of an item put up for sale, seller characteristics, buyer characteristics and the APIs of the logical components are the same as in PA2. Also, the communication mechanisms between client and frontend (REST), frontend and backend (gRPC), and frontend and third-party service (SOAP/WSDL) are the same.

A key limitation of the online marketplace system we have built so far is that the server (frontend and backend) is both a performance bottleneck and a failure bottleneck. In this

assignment, we will address this limitation by replicating all logical components of server frontend and backend over different hosts.

Requirements of PA3

Design and Implement a Rotating Sequencer Atomic Broadcast Protocol

In this atomic broadcast protocol, all requests are first broadcast to all replicas. Global ordering is then assigned by a rotating sequencer: the node responsible for sequence number k chooses one pending request that has not yet been assigned a global sequence number, assigns it sequence number *k,* and broadcasts a *Sequence* message. All replicas then deliver requests in global sequence order. You will first need to work out the details of this protocol and then implement it.

The protocol has three types of messages: *Request message*, *Sequence message,* and *Retransmit request message*.

Assume that a group has *n* members, each having a unique ID, *0* ... *n-1*. A client may submit an application request to any one of the group members. On receiving an application request from a client, a group member sends a *Request message* to every group member. This message includes a unique request ID <*sender_ID*, *local_seq_num*>, the client request, and *some additional metadata*. The *local seq number* included in the Request message is derived from a monotonically increasing counter that each group member maintains locally (initial value 0, incremented after sending each new Request message).

For each Request message, one of the group members assigns a global sequence number to this Request message and sends out a *Sequence message* to all group members. This message includes the assigned global sequence number, request ID of the Request message and *some additional metadata*. The global sequence numbers are assigned in a (globally) monotonically increasing order starting from *0*. The global sequence number assigned to a Request message determines the delivery order of that Request message.

The task of assigning global sequence numbers and sending Sequence messages is shared by all group members as follows: A Sequence message with global sequence number *k* is sent out by the group member whose member ID is *k mod n*. This group member assigns this global sequence number to a Request message (with message ID <*sender_ID, local_seq_num>)* that it has received, but has not yet been assigned a

global sequence number. If the Sequencer does not yet have an available message, it waits until one arrives. It sends out a Sequence message under the following conditions:

1. It has received all Sequence messages with global sequence numbers less than *k.*
2. It has received all Request messages to which global sequence numbers less than *k* have been assigned.
3. All Request messages sent by the member with ID *sender_ID* and local seq numbers less than *local_seq_num* have been assigned a global sequence number.

A group member delivers a Request message with assigned global sequence number *s* to the application only after:

1. It has delivered all Request messages with assigned global sequence numbers less than *s.*
2. It has ensured that a majority of the group members have received all Request messages as well as their corresponding Sequence messages with global sequence numbers *s* or less.

Group members use negative acknowledgement to recover from message losses. They send a *Retransmit request message* whenever they detect a missing Request message or a Sequence message. The Retransmit request message is sent to the sender of the missing message. The Retransmit message may also contain *some metadata*.

Your task is to determine what metadata to include in the Request, Sequence and Retransmit request messages. The goal of these metadata is to (1) determine whether the delivery conditions of a Request message have been satisfied, and (2) detect missing Request or Sequence messages. Useful metadata might include the highest sequence number received/delivered by a node or the highest request received from each sender.

Finally, group members should use UDP as the underlying communication protocol for all communication.

Replication of Customer Database

Replicate the customer database over five servers using your rotating sequencer atomic broadcast protocol. Assume that these servers do not fail, however communication is unreliable.

Replication of Product Database

Replicate the product database over five servers using Raft. Download an implementation of Raft from the Internet. There are several open-source implementations available in different languages (See https://raft.github.io/). Assume that the servers can suffer crash failures and communication is unreliable.

Replication of Server-Side Seller's Interface and Buyer's Interface

Replicate the server-side seller's interface and server-side buyer's interface on four servers each. Since these components are stateless, there is no need for any protocol to manage these replications. Configure your client-side sellers interface and client-side buyers interface with a list of all replicas and let the client connect to one of them. In case a client gets disconnected from a replica it was connected to, it would try to connect to another replica from the list. In case one of the replicas suffers crash failure, just restart it (ensure that it has the same IP address).

Evaluation

The evaluation for PA3 is similar to before. We define the following metrics that you should be able to measure:

- *Average response time*: Response time of a client-side API function is the duration of time between the time when the client invokes the function and time when a response is received from the server. To measure average response time, measure the response time for ten different runs and then take the average.
- *Average server throughput*: Throughput is the number of client operations completed at the server per second. To measure average throughput, measure the throughput for ten different runs and then take the average. Each run should consist of each client invoking 1000 API functions.

For this assignment, run your server components, including all replicas, as separate instances on cloud. Ideally, each replica should run on its own VM. However, if you do not have access to enough VMs, you may run multiple processes on the same VM (e.g., using different port numbers). Your deployment should use at least 4 VMs. In this case, spread replicas of the same component across different VMs to simulate a distributed workload. Clearly document your deployment setup in the README. Each replica must run as a separate process and communicate using the network stack, even if multiple replicas share the same VM.

Report the following performance numbers for three different scenarios. You may manually / via a script trigger a failure by terminating a replica.

- Average response time for each client function when all replicas run normally (no failures).
- Average response time for each client function when one server-side sellers interface replica and one server-side buyers interface to which some of the clients are connected fail.
- Average response time for each client function when one product database replica (not the leader) fails.
- Average response time for each client function when the product database replica acting as leader fails.

The three scenarios are:

- Scenario 1: Run one instance of seller and one instance of buyer.
- Scenario 2: Run ten instances of buyers and ten instances of sellers concurrently.
- Scenario 3: Run 100 instances of buyers and 100 instances of sellers concurrently.

Provide explanations/insights for the performance you observe for each of the three scenarios. Provide an explanation for the differences in the performances you observe among the three scenarios. Finally provide a comparison between performance in your PA2 and PA3 implementations. Provide an explanation for the differences in the performances you observe.

Submission

Submit a single *.zip* file that contains all source code files, deployment files, a README file and a performance report file to Gradescope. In the README file, provide a brief description of your system design along with any assumptions (8-10 lines) and the current state of your system (what works and what not). In the performance report file, report all performances measured along with your explanation.